

DHT11/22

Sensor de humedad y temperatura

Interfaces de HW y SW

MicroPython

1. Objetivo

Implementar el sensor DHT11 utilizando MicroPython en el ESP32-C6 para obtener mediciones de temperatura y humedad relativa del ambiente, visualizando los datos en la terminal serial.

1.1 Objetivos Específicos

- Identificar las características y el principio de funcionamiento del sensor DHT11 para la medición de temperatura y humedad relativa.
- Realizar la conexión física del sensor DHT11 al ESP32-C6, verificando el correcto cableado de alimentación y comunicación.
- Implementar un programa en MicroPython que permita establecer la comunicación entre el microcontrolador y el sensor.
- Adquirir datos de temperatura y humedad relativa mediante las funciones proporcionadas por la librería dht.

1.2 Alcance de la Práctica

La presente práctica abarca la implementación básica del sensor DHT11 utilizando un microcontrolador ESP32-C6 programado en MicroPython. Se contempla la conexión física del sensor, la configuración del pin de comunicación, la adquisición de datos de temperatura y humedad relativa, así como la visualización de las mediciones en la terminal serial.

No forma parte del alcance de esta práctica la calibración del sensor, el almacenamiento de datos, la transmisión inalámbrica de información, el análisis estadístico de las mediciones ni el desarrollo de sistemas de control basados en las variables adquiridas. Estas aplicaciones podrán abordarse en prácticas posteriores utilizando los conocimientos obtenidos durante esta actividad.

2. Hardware Utilizado

2.1 Componentes

Componente	Cantidad	Descripción
ESP32-C6	1	Microcontrolador principal con soporte I2C, WiFi y BLE
Sensor DHT 11	1	sensor digital de temperatura y humedad relativa.

El DHT11 es un sensor digital diseñado para medir **temperatura y humedad relativa** del ambiente. Integra un sensor capacitivo para humedad, un termistor para temperatura y un microcontrolador interno encargado de procesar y transmitir los datos en formato digital.

El DHT11 utiliza un protocolo digital propietario conocido como:

- Single Wire Serial Interface

Aunque utiliza una sola línea de datos, es un protocolo propio del fabricante.

2.2 Secuencia de comunicación del sensor DHT11

1. **Estado de reposo**
 - La línea de datos (DATA) permanece en estado lógico alto gracias a una resistencia pull-up conectada a VCC.
 - El sensor espera una solicitud de lectura por parte del microcontrolador.
2. **Señal de inicio del microcontrolador**
 - El ESP32-C6 coloca la línea DATA en nivel bajo durante al menos 18 ms.
 - Esta señal indica al DHT11 que se desea realizar una nueva medición.
3. **Liberación de la línea de datos**
 - El microcontrolador cambia el pin a entrada y libera la línea.
 - La resistencia pull-up devuelve la línea al nivel lógico alto.
4. **Respuesta del DHT11**
 - El sensor responde colocando la línea DATA en nivel bajo durante aproximadamente 80 μ s.
 - Posteriormente coloca la línea en nivel alto durante otros 80 μ s.
 - Esta secuencia indica que el sensor ha recibido correctamente la solicitud y está listo para transmitir.
5. **Inicio de la transmisión de datos**
 - El DHT11 comienza a enviar los datos correspondientes a humedad y temperatura.
 - La información se transmite en un total de 40 bits.
6. **Transmisión de cada bit**
 - Cada bit inicia con un pulso de aproximadamente 50 μ s en nivel bajo.
 - Después del pulso bajo, la duración del nivel alto determina el valor transmitido:
 - Entre 26 y 28 μ s → Bit '0'
 - Aproximadamente 70 μ s → Bit '1'
7. **Envío de los cinco bytes de información**
 - Byte 1: Parte entera de la humedad relativa.
 - Byte 2: Parte decimal de la humedad relativa.
 - Byte 3: Parte entera de la temperatura.
 - Byte 4: Parte decimal de la temperatura.
 - Byte 5: Checksum o suma de verificación.
8. **Verificación de integridad**
 - El microcontrolador suma los primeros cuatro bytes recibidos.
 - El resultado debe coincidir con el byte de checksum enviado por el sensor.
 - Si los valores coinciden, la trama se considera válida.
9. **Procesamiento de los datos**
 - El microcontrolador extrae los valores de temperatura y humedad.
 - Los datos quedan disponibles para su visualización o procesamiento.
10. **Retorno al estado de reposo**
 - Finalizada la transmisión, la línea DATA vuelve al estado lógico alto.
 - El sensor permanece a la espera de una nueva solicitud de lectura.

2.3 Mapa de Pines

Variable	GPIO ESP32-C6	Resistencia Interna	Función
VCC	5V	NA	Voltaje de alimentación
Datos	GPIO 4	NA	Canal de datos
GND	GND	NA	Referencia de voltaje

2.4 Esquema de Conexión

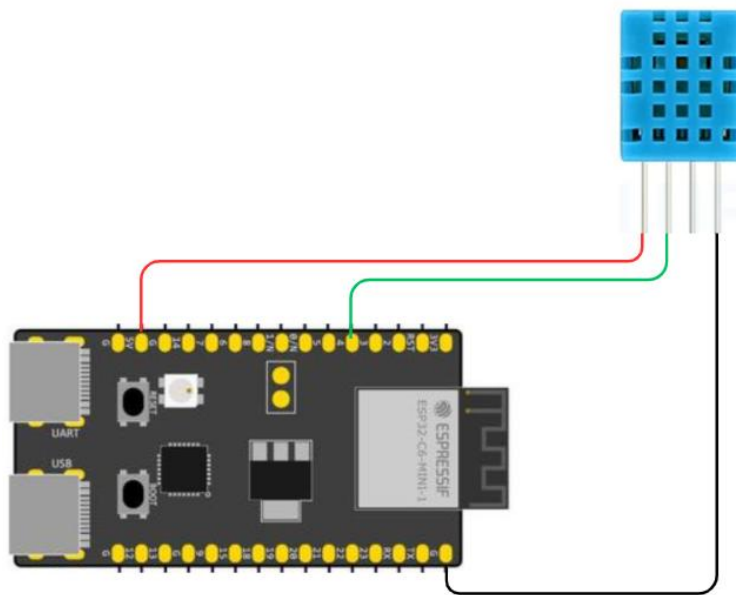


Figura 1. Esquema de conexión.

2.5 Pruebas Eléctricas Realizadas

Antes de cargar el firmware se realizaron las siguientes verificaciones eléctricas para garantizar la integridad del circuito, el correcto funcionamiento de los dispositivos de entrada y salida, y prevenir daños al microcontrolador y/o sensor:

- **Verificación de la alimentación del sensor:** Se midió el voltaje entre los terminales VCC y GND del DHT11 para comprobar que el sensor recibía la tensión de alimentación adecuada.
- **Verificación del estado lógico del pin de datos:** Se comprobó que la línea de datos permaneciera en estado alto cuando no existía transmisión, confirmando el funcionamiento correcto de la resistencia pull-up incorporada en el módulo.

- **Validación de la lectura de temperatura:** Se observaron los valores de temperatura reportados por el sensor para comprobar que fueran coherentes con las condiciones ambientales del laboratorio.
- **Prueba de estabilidad de las mediciones:** Se realizaron lecturas consecutivas con intervalos de dos segundos para evaluar la repetibilidad y consistencia de los datos adquiridos.

3. Descripción Técnica Detallada

El programa tiene como finalidad establecer la comunicación entre el ESP32-C6 y el sensor DHT11 para realizar mediciones periódicas de temperatura y humedad relativa, mostrando los resultados en la terminal serial. Para facilitar su comprensión, el código se divide en las siguientes etapas:

3.1 inclusión de librerías

En esta etapa se importan las librerías necesarias para acceder a los periféricos del microcontrolador y a los módulos externos.

```
import machine
import dht
import time
```

- **machine:** Permite acceder y configurar los recursos hardware del microcontrolador, como los pines GPIO.
- **dht:** Contiene las funciones necesarias para establecer la comunicación con sensores de la familia DHT.
- **time:** Proporciona funciones relacionadas con temporización y retardos.

3.2 Configuración del pin de comunicación

Se configura el GPIO 4 del ESP32-C6 como pin de comunicación con el sensor DHT11.

La variable `datos` almacena la referencia al pin físico que será utilizado para transmitir y recibir información mediante el protocolo digital del sensor.

De igual manera se crea una instancia de la clase DHT11, asociándola al pin previamente configurado.

A partir de este momento, la variable `sensor` permite acceder a todas las funciones necesarias para interactuar con el dispositivo, como iniciar una medición y obtener los valores de temperatura y humedad.

```
datos = machine.Pin(4) # Configuración del pin de datos
sensor = dht.DHT11(datos) # Crea la instancia del sensor
```

3.3 Bucle Principal

El programa espera dos segundos antes de realizar una nueva medición.

Este retardo es necesario debido a las limitaciones del DHT11, ya que el fabricante recomienda no realizar lecturas consecutivas demasiado rápidas.

Durante este tiempo el sensor actualiza internamente sus mediciones.

La función `measure()` inicia el proceso de adquisición de datos.

Durante esta operación:

1. El ESP32-C6 envía una señal de inicio al DHT11.
2. El sensor responde indicando que está listo para transmitir.
3. Se envían los datos correspondientes a temperatura y humedad.
4. La librería decodifica la información recibida y la almacena internamente.

Esta es la instrucción más importante del programa, ya que sin ella no se actualizan las mediciones.

Se recupera el valor de temperatura obtenido durante la última medición realizada.

El dato se devuelve en grados Celsius y se almacena en la variable `temperatura`.

Se obtiene el porcentaje de humedad relativa medido por el sensor.

El valor se almacena en la variable `humedad`.

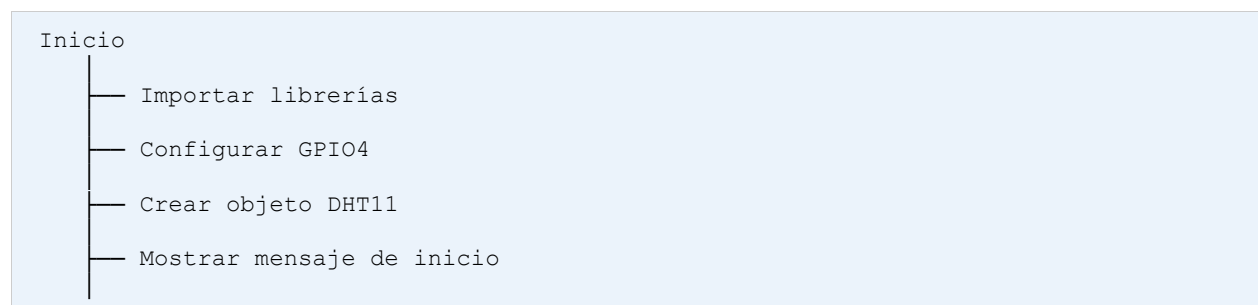
```
while True:
    try:
        time.sleep(2) # Estabilizacion del DTH11
        sensor.measure() # Realiza la Medicion
        temperatura = sensor.temperature() # Celsius
        humedad = sensor.humidity()      # Porcentaje %

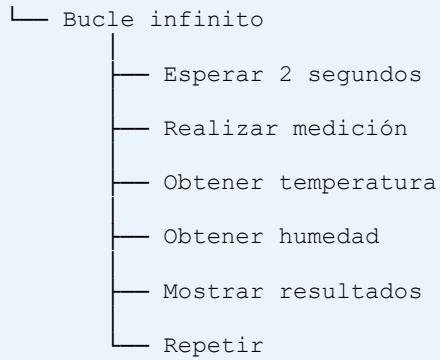
        print(f"Temperatura: {temperatura}°C | Humedad: {humedad}%")

    except OSError as e:
        print("Error al leer el sensor. Revisa las conexiones.")
```

4. Flujo General del Sistema

El siguiente diagrama describe el flujo de ejecución completo del firmware desde el arranque:





5. Conclusiones

En la presente práctica se logró implementar exitosamente el sensor DHT11 utilizando un ESP32-C6 programado en MicroPython, permitiendo la adquisición y visualización de datos de temperatura y humedad relativa en tiempo real. Durante el desarrollo se comprendió el proceso de conexión y configuración del sensor, así como el uso de la librería dht para simplificar la comunicación mediante su protocolo digital de un solo hilo.

Además, se verificó el correcto funcionamiento del sistema mediante diversas pruebas eléctricas y de operación, observando que las mediciones obtenidas fueron consistentes con las condiciones ambientales presentes.

6. Referencias

Aosong Electronics. DHT11 Temperature and Humidity Sensor Datasheet.

MicroPython DHT Module Documentation. Disponible en:
<https://docs.micropython.org/en/latest/esp8266/tutorial/dht.html>

Adafruit Industries. DHT11, DHT22 and AM2302 Sensors Guide. Disponible en:
<https://learn.adafruit.com/dht>

7. Anexos

```
import machine
import dht
import time

datos = machine.Pin(4) # Configuracion del pin de datos
sensor = dht.DHT11(datos) # Crea la instancia del sensor

print("Iniciando lectura de datos...")

while True:
    try:
        time.sleep(2) # Estabilizacion del DTH11
        sensor.measure() # Realiza la Medicion
        temperatura = sensor.temperature() # Celsius
        humedad = sensor.humidity() # Porcentaje %

        print(f"Temperatura: {temperatura}°C | Humedad: {humedad}%")

    except OSError as e:
        print("Error al leer el sensor. Revisa las conexiones.")
```
